

Format for describing dependencies of source files

Ben Boeckel, Brad King

<ben.boeckel@kitware.com, brad.king@kitware.com>

version P1689R5, 2022-06-03

Table of Contents

[1. Abstract](#)

[2. Changes](#)

[2.1. R5 \(pending\)](#)

[2.2. R4 \(June 2021 mailing\)](#)

[2.3. R3 \(Dec 2020 mailing\)](#)

[2.4. R2 \(pre-Prague\)](#)

[2.5. R1 \(post-Cologne\)](#)

[2.6. R0 \(Initial\)](#)

[3. Introduction](#)

[4. Motivation](#)

[4.1. Why Makefile snippets don't work](#)

[5. Assumptions](#)

[6. Format](#)

[6.1. Schema](#)

[6.2. Storing binary data](#)

[6.3. Filepaths](#)

[6.4. Rule items](#)

[6.5. Module dependency information](#)

[6.5.1. Language-specific notes](#)

[Fortran](#)

[C++](#)

[6.6. Extensions](#)

[7. Versioning](#)

[8. Full example](#)

[9. References](#)

Document number ISO/IEC/JTC1/SC22/WG21/P1689R5

Date 2022-06-03

Reply-to Ben Boeckel, Brad King, ben.boeckel@kitware.com, brad.king@kitware.com

Audience EWG (Evolution), SG15 (Tooling)

1. Abstract

When building C++ source code, build tools need to discover dependencies of source files based on their contents. This occurs during the build so that file contents can change without build tools having to reinitialize and so the dependencies of source file generated during the build are correct. With the

advent of modules in [P1103R3], there are now ordering requirements among compilation rules. These also need to be discovered during the build. This paper specifies a format for communicating this information to build tools.

2. Changes

2.1. R5 (pending)

Changed:

- Added another example which imports the same header in various ways.
- Add an `is-interface` key to indicate whether a provided module is visible or not.

2.2. R4 (June 2021 mailing)

Changed:

- Removed `work-directory` from the list of required keys. It is not strictly necessary because the build system should be controlling working directories in a way it understands anyways.
- Removed `inputs`, `outputs`, `depends` from rules. These were intended to replace depfiles (i.e., `-M` flags in `gcc`), but for module dependencies, implementation experience showed that parsing this information was extraneous work for tools that just needed the module dependency information. Future proposals can tackle this problem and, in order to keep the information outside of this file, may be referenced by a new key to be added in a future version (ideally in a backwards-compatible way).
- `future-compile` is now folded into its parent object as it is the only one left.
- The `primary-output` key is added to rules in order to know which rule it is associated with. Without it, there was an ambiguity of which `outputs` item was the primary output. This is analogous to the `-MT` flag in `gcc` to control the output name of the `make` snippet in the depfile.
- Introduce `unique-on-source-path` in order to disambiguate header units for C++.
- Required module descriptions now contain a `lookup-method` property which describes how the module was requested by the source. It defaults to `by-name`.
- Specify defaults for `provides` and `requires` as empty arrays.
- Improve examples.

2.3. R3 (Dec 2020 mailing)

Changed:

- `work-directory` is now represented per-rule rather than as a top-level entry.
- arbitrary binary data format storage has been removed. This was deemed as too complicated for the benefits gained at this time. If experience shows that the generalizations are needed, the binary representation can be revisited in the future.
- the example filenames are now consistent with each other.

2.4. R2 (pre-Prague)

Added:

- more background information (motivation and assumptions)
- validity of source entries depends on uniqueness in the outputs, not the inputs.
- `input` is now an array, `inputs`. This is to support representation of unity builds where multiple input sources are compiled at once into a single set of outputs.
- `sources` is renamed to `rules`. There is not necessarily a 1-to-1 correlation with source files with compilation rules.
- full example output for a C++ source
- uniformly use "property" instead of "key" for JSON fields.

2.5. R1 (post-Cologne)

The following changes have been made in response to feedback in SG15:

- rename keys to be more "noun-like" or for clarity including:
 - `readable` → `readable-name`
 - `logical` → `logical-name`
 - `filepath` → `compiled-module-path`
- remove `future-link` (no known use case)
- remove %-encoding for filepaths
- remove top-level `extensions` key (still possible, just use `_` keys)
- require vendor prefixes for extensions
- add an optional `source-path` key to `depinfo` objects

The following changes have been made in response to feedback in SG16:

- change the name of the "data" key to "code-units"
- mention normalization for JSON encoders and decoders

2.6. R0 (Initial)

Description of the format and its semantics.

3. Introduction

This paper describes a format designed primarily to communicate dependencies of source files during the building of C++ source code. While other uses may exist, the primary goal is correct compilation of C++ sources. The tool which generates this format is referred to as a "dependency scanning tool" in this paper.

The contents of this format includes:

- the resources that will be required to exist when the scanned translation unit is compiled; and

- the resources that will be provided when the scanned translation unit is compiled.

This information is sufficient to allow a build tool to order compilation rules to get a valid build in the presence of C++ modules.

4. Motivation

Before C++ modules, the only kinds of dependencies on files that a build system would care about could be determined during the execution of that rule. This is because each compilation was independent of other compilation rules. However, with modules, compilation rules can now depend on each other and they must be executed in order. Build tools need to be able to extract this information from source files before compiling them due to this new ordering requirement.

Incidentally, this is exactly analogous to the problem that Fortran build systems has with Fortran modules. To that end, this format is explicitly not specific to C++ and is intended for use within the Fortran ecosystem in the future. Terminology specific to C++ is avoided in this format to avoid any indications that it is C++-specific.

4.1. Why Makefile snippets don't work

Historically, dependency information of a build rule has been handled by Makefile snippets. An example of this is:

```
output: input_a input_b input_c
```

This states that the artifact of the build rule is `output` and files `input_a`, `input_b`, and `input_c` were read during its creation. This allows the build system to know that if any of the listed `input_*` files changes, the rule for `output` needs to be brought up-to-date as well.

This works decently well for the kinds of dependencies that have occurred in C++ to date, namely header includes. This is because these dependencies can be discovered while executing the rule associated with `output`.

The issue that arises with the Makefile design is that modules are a new kind of dependency that cannot be represented in declarative Makefile syntax. For example, GCC outputs variable modifications (`CXX_MODULES+=...`) into these snippets which is commonly not supported by the consuming tools. In addition, because these dependencies must be discovered **before** the compilation rule is executed, there would need to be one rule that writes dependency information for another.

As an example of the restrictions placed on these Makefile snippets, the `ninja` [\[ninja\]](#) build tool requires that `output` be the same for the rule which wrote out the dependency snippet and that no other outputs are mentioned. No other Makefile syntax is supported (variables, adding rules, special variables, macro expansions, etc.). This is because `ninja` is reading these for just the dependency information.

5. Assumptions

This format assumes the following things about the environment in which it is used: uniformity of the environment between creation and usage; only used within one build of a project; it does not apply to different configurations of a build (since dependencies may vary with the target platform or build settings such as whether it is debug or not).

It is generally assumed that the environment in which a file of this format is created is the same as the environments in which it will be read and ultimately used during the actual compilation. However, build systems may have different strategies for executing rules and when this is the case, it is assumed that the build system itself knows how to translate between the environments it sets up for each rule. For example, a build system which distributes the builds across multiple machines (whether over a network or using containerization) should know how to translate between the environment set up for one execution and another execution.

Environments can have many knobs which change fundamental behaviors of the system. A non-exhaustive list includes:

- mount layout (particularly of the input and output absolute paths)
- encoding (active code page, locale)
- effective permissions (process user and group, security modules, anti-virus)

The first two can be translated between different rules in a straightforward way. For example, if one rule is executed in a `/chroot/exec1` prefix while another is under `/chroot/exec2`, it is assumed that the build system constructed those environments and knows that paths underneath those prefixes should be rerooted for another execution rule to get its paths correct. Encoding differences can be converted between using either system APIs or libraries which handle encodings. If there are permission differences between the scanner and the compiler, it is hard to imagine how a build tool would be able to translate the file effectively.

Given that there are various things that can interfere with interpretation of the files, it is recommended that the producers will provide mechanisms to add extra context in order for build systems to be able to link up various bits together. For example, `work-directory` may be obtained via a call such `getcwd()`, but if the build tool knows that is not going to be correct (e.g., due to chroot mechanisms), tools should provide ways of specifying what this path is.

Also note that, due to the way environments can be set up, it is assumed that `..` is interpreted as the operating system does. Namely, symlinks may alter behavior from what is expected based solely on the manipulation of the path as a string.

6. Format

The format uses JSON [\[ECMA-404\]](#) as a base for encoding its information. This is suitable because it is structured (versus a plain-text format), parsers for JSON are readily available (versus candidates with a custom structural format), and the format is simple to implement (versus candidates such as YAML or TOML) which will allow for easy adoption.

JSON specifies that documents are Unicode. However, due to the way filepaths are represented in this format, it is further constrained to be a valid UTF-8 sequence.

6.1. Schema

For the information provided by the format, the following JSON Schema [\[JSON-Schema\]](#) may be used.

► JSON Schema for the format

6.2. Storing binary data

This format uses UTF-8 as a communication channel between a dependency scanning tool and a build tool, but filepath encodings are specific to the platform which means considerations for paths containing

non-UTF-8 sequences must be made. However, the most common uses of paths and filenames are either valid UTF-8 sequences or may be unambiguously represented using UTF-8 (e.g., a platform using UTF-16 for its path APIs has a valid UTF-8 encoding), so requiring excessive obfuscation in all cases is unnecessary.

After discussion with stakeholders, complicating the format for corner cases of filepaths which do not have unambiguous UTF-8 representations is an unnecessary complication at the moment. Future versions of the format may have a way to unambiguously transmit filepaths that are not Unicode-unambiguous or not valid Unicode if the need arises..

There are some use cases (though rare) which cannot be handled without a way to represent arbitrary paths. These include (but are not limited to):

- Windows paths with unpaired surrogate half codepoints (for which there is no valid UTF-8 representation).
- Encodings historically used for East Asian languages including Big-5, SHIFT-JIS, and others. There are characters in these encodings which share a Unicode representation, so there is no lossless way to use UTF-8 strings as a transport for these paths.

These restrictions have been deemed to not be important enough to support at this time in the general format. Note that many build tools already have restrictions in characters due to implementation details. For example, Makefiles have trouble representing paths ending with a `\\` character and CMake has issues with paths containing its list separator, the semicolon.

6.3. Filepaths

Filepaths may be either relative or absolute. Build tools generally already know the working directory because it chooses where to execute the tool in the first place. If other tools require this information, it should be indicated to the program writing the format which then shall write the provided directory to the format in the `work-directory` property.

6.4. Rule items

The `rules` array allows for the dependency information of multiple rules to be specified in a single file.

The only restriction on the contents of the collective set of `rules` objects is that the set of all `primary-output` and `outputs` of each object must be unique. This is because if they are not unique, there are outputs which have multiple rules that write to them, which is, in general, undefined behavior in build tools.

6.5. Module dependency information

Each rule represented in the `rules` array is a JSON object which has four optional properties, `primary-output`, `outputs`, `provides`, and `requires`. They are described as follows:

- `primary-output` (optional): The primary output of the rule being scanned. The build system generally dictates this value since it has to know what each rule outputs in order to make a build graph. To support this, tools writing this format should provide a way to fill in this value and it shall be written to this property.
- `outputs` (optional): An array of additional outputs created by the rule being scanned.
- `provides` (optional; defaults to `[]`): An array of module description objects which are provided by the rule being scanned.

- **requires** (optional; defaults to `[]`): An array of module description objects which are required in order to perform the rule being scanned.

Module descriptions for the **provides** and **requires** arrays include the following properties:

- **logical-name** (required): The name of the module. This is the name to use to correlate between providing and requiring modules in order to determine the order compilation rules need to be performed.
- **compiled-module-path** (optional): The path to the compiled module (if known).
- **source-path** (optional if **unique-on-source-path** is **false**): The path to the source file for this module (if known).
- **unique-on-source-path** (optional; defaults to **false**): Whether the module is unique based on its **logical-name** (if **false**) or **source-path** (if **true**).

Additionally, those in the **requires** array may also include a property indicating how the module was found:

- **lookup-method** (optional; defaults to **"by-name"**): Must be one of **by-name**, **include-angle**, or **include-quote**.

Those in the **provides** array also include a property indicating whether the provided module contributes to an interface:

- **is-interface** (optional; defaults to **true**): Whether the provided module contributes to the interface or not. What this means is language-dependent.

6.5.1. Language-specific notes

Fortran

Fortran only supports named modules, so it will generally never have use the **include-lookup-method** values. Additionally, the **source-path** will generally not be known either as Fortran compilers only support finding compiled modules. Fortran also supports providing multiple modules from a single source file which is the reason **provides** is an array rather than a single object.

C++

In C++20, source files may only provide a single module or module partition, so only one will be provided for each rule. Some compiler may choose to implement the **:private** module partition as a separate module for lookup purposes, and if so, it should be indicated as a separate **provides** entry.

A module (partition or not) which uses the **export** keyword would have its **is-interface** key set to **true** otherwise it would be **false**.

For header units, the logical name will likely be the full, normalized path to the header itself in order to correlate its usage no matter how it is included in the literal source. This is because a header imported via **import "header.h";** or **import <parent/header.h>;** should both resolve to the same module. In such cases, **unique-on-source-path** should be set to **true**. This will allow the build system to know that the **"header.h"** and **<parent/header.h>** modules are both the same.

In some cases, it may be important to know whether the module was imported via angle brackets or quotes. For this case, the **logical-name** can drop the decoration and instead specify **lookup-method** to the appropriate indicator.

Example source entry

```
{
  "primary-output": "path/to/output.o",
  "provides": [
    {
      "compiled-module-path": "exported.bmi",
      "logical-name": "exported"
    }
  ],
  "requires": [
    {
      "logical-name": "imported"
    }
  ]
}
```

6.6. Extensions

Vendor extensions may be added to the format using properties prefixed with an underscore (`_`) followed by a vendor-specific string followed by another underscore. None of these may be used to store semantically relevant information required to execute a correct build. Consumers must be able to ignore all `_`-prefixed properties and not suffer any loss of essential functionality.

Example source entry with extended information

```
{
  "primary-output": "path/to/output",
  "_VENDOR_extension": true
}
```

7. Versioning

There are two properties with integer values in the top-level JSON object of the format: **version** and **revision**. The **version** property is required and if **revision** is not provided, it can be assumed to be `0`. These indicate the version of the information available in the format itself and what features may be used. Tools creating this format should have a way to create older versions of the format to support consumers that do not support newer format versions.

The **version** integer is incremented when semantic information required for a correct build is different than the previous version. When the **version** is incremented, the **revision** integer is reset to `0`.

The **revision** integer is incremented when the semantic information of the format is the same as the previous revision of the same version, but it may include additionally specified information or use an additionally specified format for the same information. For example, adding a **modification_time** or **input_hash** field may be helpful in some cases, but is not required to understand the dependency information. Such an addition would cause an increment of the **revision** value.

The version specified in this document is:

Version fields for this specification


```
{
  "version": 1,
  "revision": 0
}
```

8. Full example

Given the following three translation units in C++.

Provide module TU (`duplicate.mpp`)

```
export module duplicate;

export int m() {
    return 0;
}
```

Provide and require modules TU (`another.mpp`)

```
export module another;
import duplicate;

export int i() {
    return m();
}
```

Require modules TU (`use.mpp`)

```
import duplicate;
import another;

int lib() {
    return m() + i();
}
```

The scanning results of each TU can be represented by:

Example scanning output

```
{
  "version": 1,
  "revision": 0,
  "rules": [
    {
      "primary-output": "duplicate.mpp.o",
      "provides": [
        {
          "logical-name": "duplicate"
        }
      ]
    }
  ]
}
```

```

    ]
  },
  {
    "primary-output": "another.mpp.o",
    "provides": [
      {
        "logical-name": "another"
      }
    ],
    "requires": [
      {
        "logical-name": "duplicate"
      }
    ]
  },
  {
    "primary-output": "use.mpp.o",
    "requires": [
      {
        "logical-name": "duplicate"
      },
      {
        "logical-name": "another"
      }
    ]
  }
]
}

```

Note that the scanner, in this case, has been told the **primary-output** for each translation unit so that the build system can correlate which rule mapping corresponds to each source file. The scanner, in this case, also does not indicate where the module files might be generated, so it does not provide **compiled-module-path** properties and it is up to the build system to generate a suitable filename for each.

An example with a header unit involved is:

Provide header unit (header .hpp)

```
#define header_unit_macro
```

Require header unit TU (use-header .mpp)

```
import <header.hpp>;

#ifdef header_unit_macro
#endif

```

The scanning results of these files can be represented by:

Example scanning output

```

{
  "version": 1,
  "revision": 0,
  "rules": [
    {
      "primary-output": "use-header.mpp.o",
      "requires": [
        {
          "logical-name": "<header.hpp>",
          "source-path": "/path/to/found/header.hpp",
          "unique-on-source-path": true,
          "lookup-method": "include-angle"
        }
      ]
    },
    {
      "primary-output": "header.hpp.bmi",
      "provides": [
        {
          "logical-name": "header.hpp",
          "source-path": "/path/to/found/header.hpp",
          "unique-on-source-path": true,
        }
      ]
    }
  ]
}

```

Note that the `logical-name` does not match because the scanner of `header.hpp` does not know how it will be imported. This is the reason that `source-path` is used and `unique-on-source-path` is set to `true`. The build system, in this case, will need to use this to inform the compilation of `use-header.mpp` to use the generated file for the header scanning under the name it will be requested during the compilation (given as `logical-name`).

Another example which imports the same header multiple ways is:

Import header multiple times (`import-many-header.mpp`)

```

import <header/dir/path.h>;
import <dir/path.h>; // Same due to a `-I.../header` entry

// Same header, different import.
import "header/dir/path.h";
import "dir/path.h";

```

The scanning results of this file could be represented by:

Example scanning output

```

{
  "version": 1,
  "revision": 0,

```

```

"rules": [
  {
    "primary-output": "import-many-header.mpp.o",
    "requires": [
      {
        "logical-name": "<header/dir/path.h>",
        "source-path": "/path/to/header/dir/path.h",
        "unique-on-source-path": true
      },
      {
        "logical-name": "<dir/path.h>",
        "source-path": "/path/to/header/dir/path.h",
        "unique-on-source-path": true
      },
      {
        "logical-name": "\"header/dir/path.h\"",
        "source-path": "/path/to/header/dir/path.h",
        "unique-on-source-path": true
      },
      {
        "logical-name": "\"dir/path.h\"",
        "source-path": "/path/to/header/dir/path.h",
        "unique-on-source-path": true
      }
    ]
  }
]
}

```

A build system should notice that only one header is actually being scanned here, but is consumed by multiple names. Whether the name that is being used to import is important or not is likely to change between implementations.

9. References

- [ECMA-404] The JSON Data Interchange Syntax. <http://www.ecma-international.org/publications/files/ECMA-ST/ECMA-404.pdf>.
- [JSON-Schema] Austin Wright and Henry Andrews. JSON Schema: A Media Type for Describing JSON Documents. <https://tools.ietf.org/html/draft-handrews-json-schema-01>.
- [ninja] Ninja, a small build system with a focus on speed. <https://ninja-build.org/>.
- [P1103R3] Richard Smith. Merging Modules. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1103r3.pdf>.
- [Unicode-12] Unicode Consortium. Unicode 12.0.0. <https://www.unicode.org/versions/Unicode12.0.0/>.